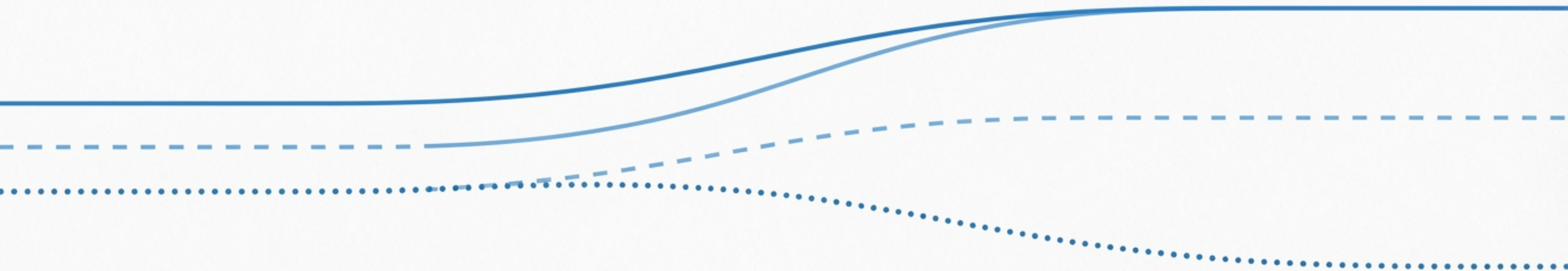


MCP 三种通信机制对比

Stdio vs. SSE vs. Streamable HTTP



本次分享大纲

1. 理解 MCP：为何需要通信协议？

Source Han Sans SC Regular

2. 机制一览：本地直连 vs. 远程服务

Source Han Sans SC Regular

3. 深度解析 1：标准输入/输出 (Stdio)

Simple & Direct Local Implementation

4. 深度解析 2：服务器发送事件 (SSE)

SSE - Lightweight & Efficient Unidirectional Push

5. 深度解析 3：可流式 HTTP

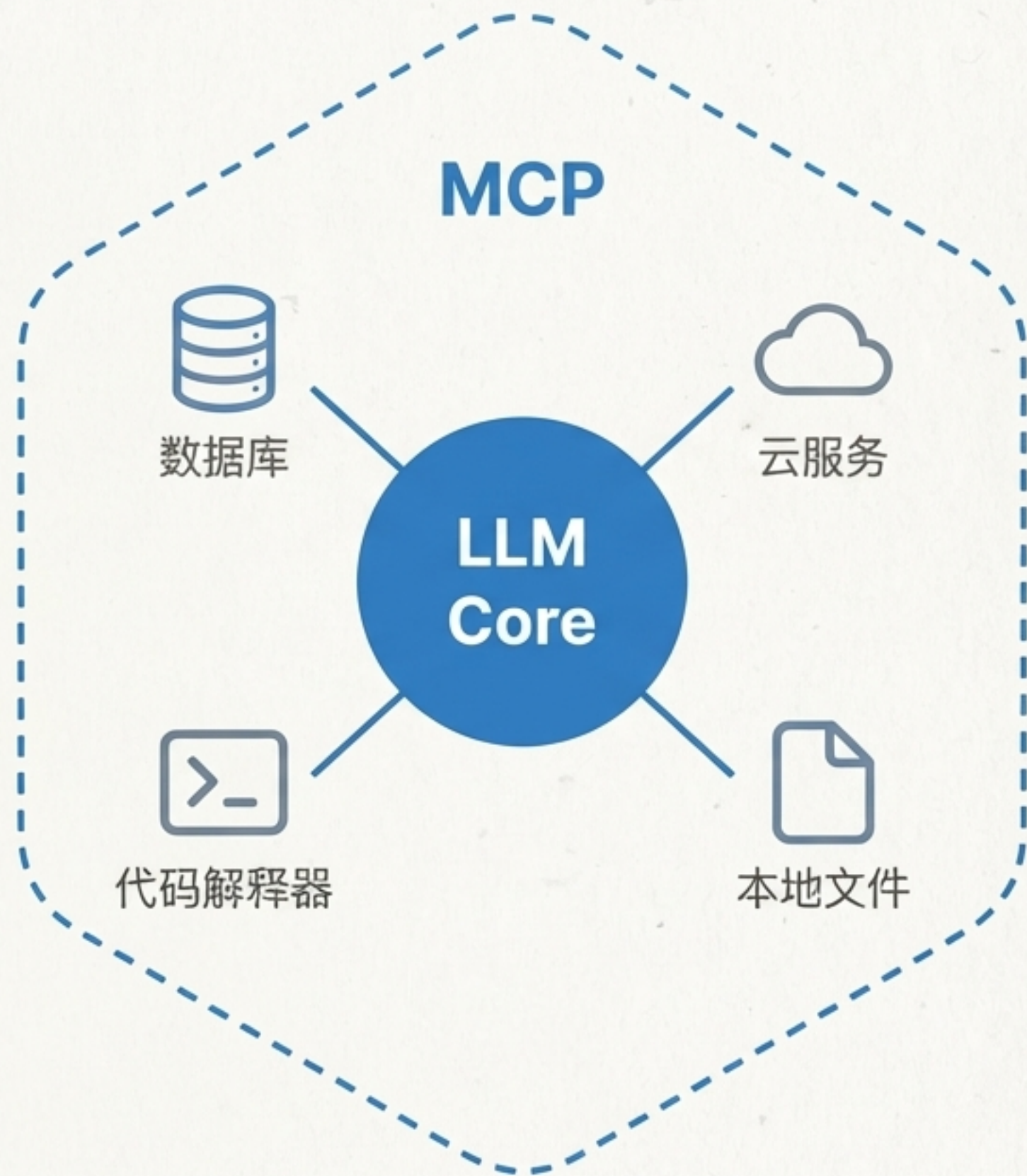
Streamable HTTP - Flexible & Powerful Bidirectional Service

6. 决策指南：如何为你的项目选择合适的机制？

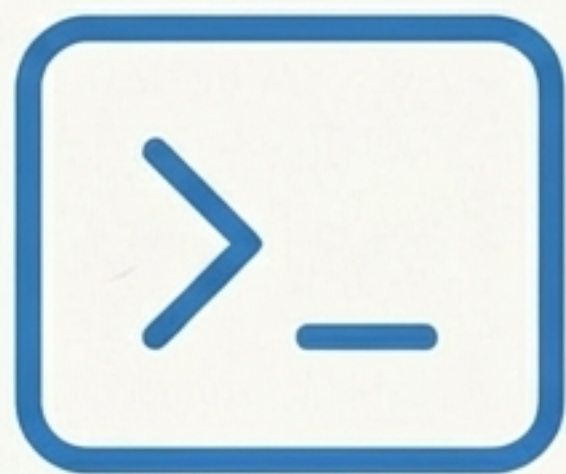
Source Han Sans SC Regular - Connertont Locat Heading

什么是 MCP? 为大模型配备一 把“瑞士军刀”

MCP (Model Context Protocol) 是一种标准化协议，旨在统一大模型与外部工具、数据、函数交互的接口。它让大模型能安全、高效地调用外部能力，就像拥有了一个功能强大的助手。



三种核心通信机制：从本地到云端



Stdio（标准输入/输出）

Source Han Sans SC Regular

用于本地开发和进程间通信，无需网络。



SSE（服务器发送事件）

Source Han Sans SC Regular

用于从服务器向客户端进行实时、单向的数据推送。



Streamable HTTP

Source Han Sans SC Regular

用于构建生产级的、可远程调用的标准网络服务。

机制一：Stdio - 零网络开销的本地集成

核心原理 (Core Principle)： 基于操作系统的标准流 (stdin, stdout) 进行数据交换。所有通信都在内存中完成，是纯粹的本地进程间通信。

核心优势 (Core Advantages)：

- ✓ **极简高效 (Simple & Efficient)：** 无需配置网络，实现简单，延迟极低。
- ✓ **绝对安全 (Absolutely Secure)：** 数据不离开本地计算机，没有网络暴露风险。

适用场景 (Use Cases)：

- 个人开发，将本地脚本或工具封装为AI助手。
- 快速原型验证和功能演示 (Quick Demos)。



Stdio 实现：一行代码完成本地工具调用

```
# main.py
from fastmcp import FastMCP
import platform, psutil, json

mcp = FastMCP("host_info_mcp")

@mcp.tool
def get_host_info() -> str:
    """获取本机系统信息"""
    info = {
        "system": platform.system(),
        "release": platform.release(),
        "processor": platform.processor(),
        "memory_gb": str(round(psutil.virtual_memory().total / (1024 ** 3), 2)),
    }
    return json.dumps(info, indent=4)

if __name__ == "__main__":
    mcp.run(transport="stdio") # 核心实现
```

切换通信机制的核心

机制二：SSE - 轻量级的服务器实时推送

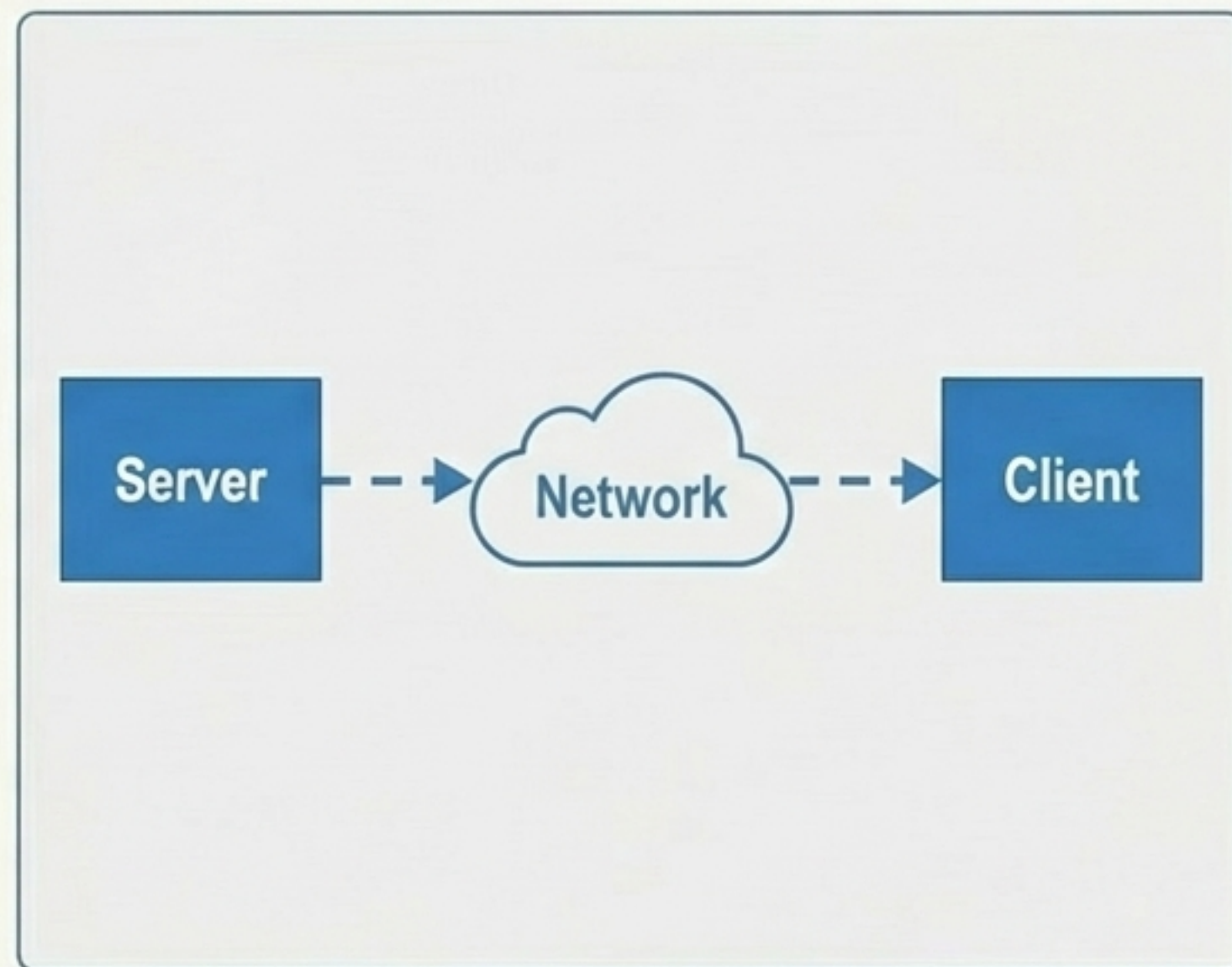
核心原理 (Core Principle)： 基于标准 HTTP 协议，允许服务器持续向客户端推送事件流。这是一种高效的单向通信模式。

核心优势 (Core Advantages)：

- ✓ **Web 友好 (Web Friendly)：** 浏览器原生支持，易于与前端应用集成。
- ✓ **轻量高效 (Lightweight & Efficient)：** 相比 WebSocket，开销更小，实现更简单。
- ✓ **内置重连 (Built-in Retry)：** SSE 标准包含自动

适用场景 (Use Cases)：

- 需要从服务端实时获取状态更新的应用，如任务进度、日志流。
- 向客户端发送实时通知。



SSE 实现：将本地工具发布为单向数据源

```
# main_sse.py
from fastmcp import FastMCP
# ... (imports and tool definition are identical)

@mcp.tool
def get_host_info() -> str:
    """获取本机系统信息"""
    # ... (function logic is identical)
    return json.dumps(info, indent=4)

if __name__ == "__main__":
    mcp.run(transport="sse") # 核心实现
```

工具逻辑不变，仅切换传输模式

机制三：Streamable HTTP - 构建生产级双向服务

****核心原理 (Core Principle) : ****

使用标准的 HTTP/HTTPS 协议，通过流式响应实现全双工、长时间的可靠通信。

****核心优势 (Core Advantages) : ****

✓ ***功能最强 (Most Powerful)** : 支持完整的客户端-服务器双向通信。

✓ ***通用性好 (Highly Versatile)** : 跨平台、跨语言，与现有网络设施（如负载均衡、网关）完全兼容。

✓ ***可扩展性强 (Scalable)** : 易于部署和横向扩展，是构建企业级服务的首选。

****适用场景 (Use Cases) : ****

- 将复杂业务逻辑（如数据库操作）封装为安全的远程工具。
- 构建可被多个不同客户端调用的、标准化的 MCP 服务。



Streamable HTTP 实现：封装一个远程数据库工具

```
# main_http.py
from fastmcp import FastMCP
import mysql.connector

mcp = FastMCP("operateMySQL")

@mcp.tool
def get_lock_tables() -> list:
    """查询当前被锁定的MySQL表"""
    # ... (database connection and query logic)
    return result

if __name__ == "__main__":
    mcp.run(
        transport="streamable-http", # 核心实现
        host="0.0.0.0",
        port=9000,
        path="/mcp"
    )
```

提供网络服务所需的额外配置 (Additional configuration)

决策指南： 如何选择最适合你的通信机制？

特性 (Feature)	Stdio (标准输入/输出)	SSE (服务器发送事件)	Streamable HTTP (可流式HTTP)
部署环境 Deployment	本地 (Local)	远程 (Remote)	远程 (Remote)
通信方式 Communication	进程间 (Inter-process)	服务器 → 客户端 (单向)	客户端 ↔ 服务器 (双向)
开发复杂度 Complexity	极低 (Very Low)	较低 (Low)	中等 (Medium)
核心优势 Key Advantage	零延迟、高安全 (Zero Latency, High Security)	轻量、Web友好 (Lightweight, Web Friendly)	功能强大、通用、可扩展 (Powerful, Versatile, Scalable)
最佳场景 Best Use Case	✓ 个人工具、本地Demo (Personal Tools, Local Demos)	✓ 实时日志、状态监控 (Real-time Logs, Status Monitoring)	✓ 生产级远程服务、数据库工具 (Production Services, DB Tools)

核心要点总结

从 `Stdio` 开始 (Start with `Stdio`)

当你只需要在本地快速集成工具时, `Stdio` 是最简单、最高效的选择。

用 `SSE` 推送 (Push with `SSE`)

当你的应用需要从服务端接收单向的实时更新时, `SSE` 是一个轻量级的标准方案。

用 `Streamable HTTP` 构建服务 (Build services with `Streamable HTTP`)

当你计划构建一个稳定、可扩展、功能完整的远程工具服务时, `Streamable HTTP` 是最可靠和功能最全面的选择。